fes-f2-gpu replay, batching bound, rootfind phase profile

User request (2026-11-14, “All”), for the three open items of `research/ic_gpu_overhead_20261114`: (1) independent replay of the `fes-f2-gpu` measurements, (2) batching several systems per GPU launch, (3) profile rootfind on a host with GPU counters.

Standing. Producer measurements below are `[pcc:unverified]` until the validator’s review (`VALIDATION_REVIEW.md`, section 4) confirms them. Wall time is a practicality note (`AGENTS.md` §6); counted S is the metric and nothing here changes it. No scoreboard or leaderboard row.

Run

One A100 80GB PCIe pod (`4x9z4e0z9orydl`, Runpod host `nwm26l3m73rj`, a host not used before), CUDA 12.8, 06:20Z–06:40Z, about $0.70, deleted. `ic` built from `fa75167c45` (Rust identical to main `9b88564250`; the only source differences are the opt-in timers below, inactive unless enabled). `job.sh`; raw output `raw/out_rp.tgz` (hash in `raw/SHA256SUMS`).

1. Replay of the end-to-end runs

The exact sweep specs of `research/ic_gpu_e2e_20261113` (`sweeps/`, byte-identical), all three engines, on a different host from current main.

Every counted field reproduces exactly in all 14 rows (3 instances, every engine and repeat, the mitm reference rows included): solver calls, counted `ops`, hit rate, and planted-log verification [pcc:unverified]. S/rho reproduces exactly on the Koblitz curve, whose calibration is pinned; on the char-2 curves it moves by −5.7% (n=25) and +6.1% (n=29) with the calibration measured once per `ic bench` run (`priced_by: measured`), by the same factor for every engine of a sweep. Wall times are lower on this host’s CPU (`inherited-f4` at n=29: 895.5 s → 662.9 s; `fes-f2-gpu`: 38.7 s → 13.5 s with the overhead changes merged since).

instance	engine	calls (orig = replay)	S/rho orig → replay	whole solve s orig → replay
Koblitz n=23, repeat 2	`inherited-f4`	1006	1628.93 → 1628.93	15.48 → 11.98
	`fes-f2`	1006	21484.69 → 21484.69	14.73 → 10.47
	`fes-f2-gpu`	1006	21484.69 → 21484.69	0.79 → 0.18
char-2 n=25	`inherited-f4`	1193	4362.09 → 4113.67	33.64 → 25.49
	`fes-f2`	2338	231367 → 218198	141.94 → 98.71
	`fes-f2-gpu`	2338	231367 → 218198	3.22 → 0.90
char-2 n=29	`inherited-f4`	8977	30856 → 32742	895.51 → 662.93
	`fes-f2-gpu`	8977	3270693 → 3470603	38.70 → 13.47

The replay was launched by the producing session, so it is a reproduction on another host, not an independent replay in the validation gate's sense; the independent part is the validator's re-derivation (section 4).

2. Batching: measured upper bound, not pursued

The descent oracle tries a target's systems one after another and keeps the first solution that lifts. Solving several systems in one launch would solve systems the sequential run never reaches, which is extra counted work; it can only save the per-call overhead around the kernel. The CUDA worker's opt-in timers (`FES_SERVE_TIMING=1`, `--serve-bin`) give that overhead directly: kernel GPU time per call (CUDA events) against the Rust side's round trip per call.

unknowns	kernel GPU µs/call	worker busy µs/call	round trip µs/call	most batching could save
24 (Koblitz n=23, repeat 2)	69.0	100.4	123.9	55 µs/call, ≤ 30% of that solve
26 (char-2 n=25)	140.9	192.6	304.8	≤ 164 µs/call, ≤ 42.4%
30 (char-2 n=29)	1272.9	1305.0	1364.9	≤ 92 µs/call, ≤ 6.1%

Worker busy and kernel GPU times are averaged over all calls of the sweep; the n=25 and n=29 round trips include one worker start-up and CUDA initialisation each (not removable by batching), so their bounds are over-estimates [pcc:unverified]. At 30 unknowns the call is 93% kernel. Batching would at best halve wall time at 24–26 unknowns, where the GPU engine's whole solve is already 22× and 109× faster (this replay; 12× and 44× with report 1's older code) than the CPU search, at the price of extra counted work. **Not implemented**; worth revisiting only if an oracle that consumes every solution of a batch (no first-hit stop) is wanted for other reasons.

3. rootfind: where the cycles go

Runpod blocks the profiler counters (`ERR_NVGPUCTRPERM`, round 2) and no other GPU provider is configured in this session, so `RF_PROFILE=1` builds sample `clock64()` at the phase boundaries of `rf_decompose_thread` (one thread in 1024, one atomic per phase; `./bench profile`). Three rounds gave the same split to 0.1 percentage point. Share of sampled cycles:

|F|, targets	lookup / chunk	probes	inversion	sweep arithmetic	load + prefix + unwind
65536, 65536	sorted / 16, 32, 64	62.3, 67.5, 66.9	17.5, 10.7, 5.6	12.9, 14.0, 14.0	7.4, 7.8, 13.6
	hash / 16, 32, 64	79.9, 83.4, 84.9	8.5, 5.1, 2.6	7.8, 7.7, 7.9	3.8, 3.8, 4.6
1024, 262144	sorted / 16, 32, 64	50.8, 58.0, 60.4	22.3, 13.9, 7.2	15.9, 18.2, 17.3	11.0, 9.9, 15.1
	hash / 16, 32, 64	41.3, 47.6, 49.5	25.5, 16.5, 10.1	20.7, 23.9, 24.7	12.6, 12.0, 15.8

Caveat. The instrumented kernel is not the production kernel: against the plain build on the same pod it runs 0.72–1.21× the time depending on the configuration (`clock64()` and the atomics change register allocation and occupancy). The plain build's first configuration in each process (sorted / 16, at both sizes; at |F|=1024 80 ms against 22 ms in earlier rounds) also carried CUDA's lazy module loading, as this job did not set `CUDA_MODULE_LOADING=EAGER`. Only chunk 64 isolates the cost of the instrumentation

itself: 1.007–1.028× (validator). The split describes the instrumented kernel; the conclusion drawn from it is coarse enough to survive that [pcc:unverified].

Reading. rootfind is bound by its table lookups, not by field arithmetic: 62–85% of the sampled cycles at |F|=65536 and 41–60% at |F|=1024. At |F|=65536 the hash lookup is a global open-addressing table with separate `keys[]` and `vals[]` arrays (two dependent random loads per probe step), and the sorted lookup is a 16-step dependent binary search. That both are bound by memory latency is an interpretation: the sampled shares say where cycles go, not why, and only hardware counters would show it. It is consistent with the fact that the launch-knob sweep and the fused sweep of round 2 could not move it. Candidates for a next round, not tried here: one 8-byte slot per entry (key fingerprint and index together, one load per probe); issuing the two ε probes of an entry together so their latencies overlap; a cache-friendlier layout (Eytzinger order) for the sorted search.

4. Independent validation

`VALIDATION_REVIEW.md` (and `VALIDATION_REVIEW.json`) were written by the repository's `validator` agent from the raw outputs with its own code, against frozen commit `95dce785b3`; the producer did not edit them. Its own format, not a `validation-gate/v1` report, and it admits no row. Summary:

- Hashes of every raw file, every planted-log certificate (76 rows recomputed independently), and the serve-protocol smoke tests (30 systems brute-forced: both workers returned the full solution sets) are confirmed.
- `research/ic_gpu_e2e_20261113/REPORT.md`: validated; the n=23 counted-S gap is 9.96×, not 9× as first written (corrected in that report).
- `research/ic_gpu_overhead_20261114/REPORT.md`: validated; its "about ±0.5%" for the char-2 S spread between runs of one arm was true of A/B 1 only (A/B 2 reaches 1.358 and 1.165), and CPU `fes-f2` moved 1.6%, not 1.5% (both corrected in that report).
- This replay: validated as a reproduction of report 1's counted results (every counted field in all 14 rows; the char-2 S/rho movement is fully explained by the re-measured price); not an independent replay in the validation gate's sense, since this session launched it. Corrections it raised on this report's draft are applied (42.4% and 6.1%, −5.7% and +6.1%, the replay's own 22× and 109×, the lazy-loading caveat at both sizes, and "memory-latency bound" marked as an interpretation).
- Not checkable from the artifacts: binary provenance (the bench JSON records no `git_commit`), unit test and local probe outputs, pod IDs and costs, report 1's quoted kernel times.
- Counted cost: confirmed unchanged by every `fes-f2-gpu` change; `inherited-f4` is cheaper at every positive price of the solver's word XORs. The reports omit the §5a price band (0, 0.1 and 1 GAE per unpriced operation), and the rho reference does not record its unpriced counters: a reporting gap, not a wrong number. With a free solver, IC stays above rho (2.79×, 2.54×, 4.08×).